

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Recurso) MEIC, MEIM	
Nº	Nome:	24/01/2024

EXAME ÉPOCA RECURSO (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 marque as frases como verdadeira (V), falsa (F) ou não responde (—). Uma opção marcada corretamente vale 0,25 valores e incorretamente desconta 0,125 valores (50% de cada alínea).

1. [1 val.] Sobre características gerais dos sistemas distribuídos

☐	O <i>Speedup</i> define a melhoria da velocidade de execução em dois sistemas com recursos diferentes
☐	A lei de <i>Amdahl</i> define que o <i>Speedup</i> de um sistema é limitado pela componente não paralelizável
☐	A elasticidade consiste em ter escalabilidade vertical (<i>scaling up</i>) e horizontal (<i>scaling out</i>)
☐	A escalabilidade num sistema, usando replicação de dados aumenta o desempenho e tolerância a falhas, mas exige a existência de um algoritmo de consenso para resolver eventuais conflitos na escrita de dados

2. [1 val.] Sobre o sistema rabbitMQ

☐	Um <i>alternate exchange</i> A associado a um <i>exchange</i> DIRECT D, só recebe as mensagens que não possam ser encaminhadas para <i>queues</i> associadas ao <i>exchange</i> D
☐	O padrão <i>work-queue</i> existe quando uma <i>queue</i> tem associados múltiplos consumidores em que todos recebem as mesmas mensagens
☐	Quando um produtor envia uma mensagem para um <i>exchange</i> não tem qualquer garantia que ela seja processada por um consumidor
☐	Se um consumidor der <i>acknowledge</i> negativo a uma mensagem, esta é sempre perdida

3. [1 val.] Na operação gRPC definida por, `rpc oper(stream MsgX) returns (MsgY)` com as mensagens: `message MsgX { int32 a = 1; }` `message MsgY { int32 b = 1; }`

☐	Sendo a operação <i>stream</i> de cliente, o servidor só poderá atender um pedido de cada vez
☐	Para criar uma mensagem do tipo <code>MsgY</code> , com valor de 10, é usado o código <code>MsgY msg = new MsgY(10);</code>
☐	A operação tem de ser invocada pelo cliente forçosamente com um <i>stub</i> não bloqueante
☐	A assinatura da operação no servidor é: <code>public StreamObserver<MsgX> oper(StreamObserver<MsgY>)</code>

4. [1 val.] Considere uma imagem Docker de nome *procFoto* com uma aplicação que processa fotos de veículos e gera novas fotos com a marca da matrícula, na diretoria `/user/fotos`

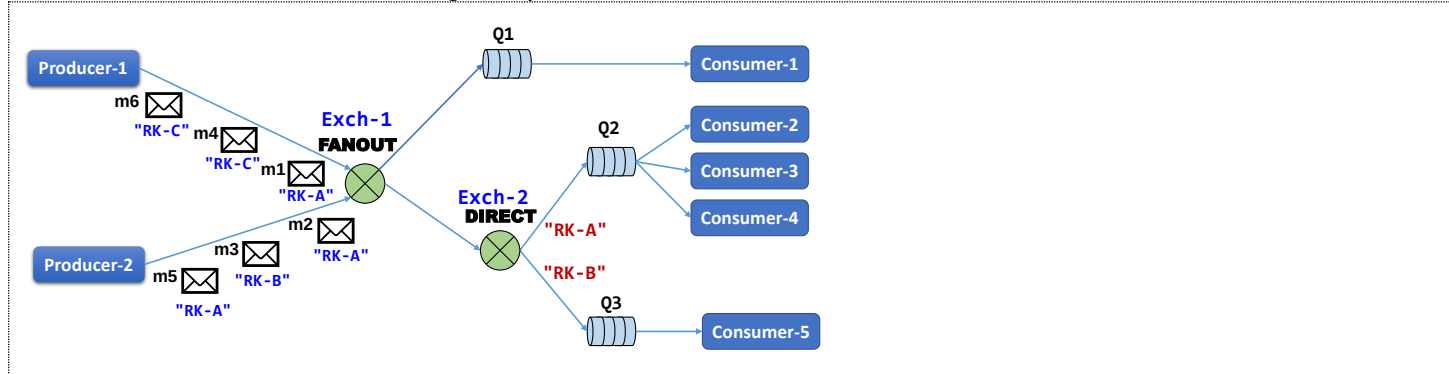
☐	Se a foto <code>car.jpg</code> estiver na máquina <i>host</i> em <code>/usr/data/car.jpg</code> podemos processar a foto executando o comando: <code>docker run -v /user/data:/user/fotos procFoto car.jpg car-marked.jpg</code>
☐	Para executar múltiplos <i>containers</i> temos de partilhar a imagem <i>procFoto</i> num sistema de ficheiros distribuídos, por exemplo o Gluster
☐	Publicando a imagem (comando PUSH) no DockerHub com o nome <code>cd/procFoto:v1</code> é possível executar posteriormente <i>containers</i> em múltiplas máquinas <i>host</i>
☐	Como a imagem processa fotos existentes na diretoria <code>/user/fotos</code> da máquina <i>guest</i> , só é possível processar fotos existentes na diretoria <code>/user/fotos</code> da máquina <i>host</i> .

5. [1 val.] Sobre algoritmos de exclusão mútua, eleição e consensos

☐	No algoritmo de consenso <i>Raft</i> existem ao longo do tempo múltiplas eleições para escolher um <i>Leader</i> que garante que cada servidor terá uma réplica dos dados consistentes
☐	O algoritmo de exclusão mútua em anel permite grande escalabilidade em número de participantes e por isso é tolerante a falhas, mesmo no caso de falha do participante que detém a permissão (<i>token</i>)
☐	As notificações de <i>membership</i> do <i>middleware</i> Spread, permitem eleger um <i>Leader</i> sem troca de mensagens entre os membros do grupo, desde que o <i>Leader</i> seja sempre o último a entrar (<i>join</i>) no grupo
☐	O algoritmo distribuído <i>Ricart&Agrawala</i> é muito ineficiente pois, para N participantes, exige $2(N-1)$ mensagens para garantir permissão de acesso em exclusão mútua a um recurso partilhado

6. [2 val.] Num sistema com comunicação *multicast* e relógios vetoriais com N participantes ($p_0, \dots, p_{N-1}$), considerando V_m o vetor da mensagem e V_k o vetor local do participante $0 \leq k \leq N-1$, as condições de verificação de ordenação causal são:
- C1) $V_m[j] = V_k[j] + 1$ j índice do processo emissor
 - C2) $V_m[i] \leq V_k[i]$ para todo $i \neq j$
- Assim se o participante p_3 ($k=3$) tiver o seu relógio local $V_3=[4,2,6,8]$, retém (*Hold*) uma mensagem, enviada por p_2 ($k=2$), com o vetor $V_m=[5,2,8,8]$ por violação das condições C1 e C2. Justifique as consequências se a mensagem não fosse retida (*Hold*) e o que terá de acontecer para que a mensagem possa ser entregue garantindo ordem causal.

7. [2 val.] Considere um sistema desenvolvido com o *middleware* RabbitMQ que permite aos produtores publicar mensagens no *exchange* Exch-1 do tipo FANOUT, o qual tem associada uma *queue* de nome Q1 e o *exchange* Exch-2 do tipo DIRECT. Considerando as *routing keys* ("RK-A", "RK-B", "RK-C") indicadas nas mensagens m1, m2, m3, m4, m5 e m6, publicadas pelo *producer-1* e *producer-2*, escreva sobre o diagrama da figura o fluxo de encaminhamento dessas mensagens, indicando claramente à frente de cada consumidor as mensagens que cada um recebe.



8. [2 val.] Considere que existem dois servidores gRPC: um servidor A (*java -jar SA.jar*) e um servidor B (*java -jar SB.jar*). O servidor A aceita pedidos no porto 8000 e o servidor B aceita pedidos no porto 9000. O servidor A, para responder aos pedidos, tem de aceder ao servidor B. Considere que existe um *script* (*run-servers.sh*) que coloca em execução os dois servidores como um único comando. Descreva como construir uma imagem Docker, que contém e inicia os 2 servidores e descreva também, como a partir dessa imagem, se podem lançar vários *containers* no mesmo *host*, expondo apenas o servidor A.